C++ to Java

A Complete Transition Guide for DSA Practitioners

This guide is written for someone who has already practiced Data Structures & Algorithms in C++ and wants to switch to Java without relearning DSA concepts from zero. It maps every C++ construct and STL container you already know to its Java equivalent, highlights the behavioral differences that trip people up, and gives side-by-side code so you can pattern-match quickly.

How to use this guide: Read it top to bottom once, then keep it as a reference while you re-solve 15-20 problems you've already done in C++, this time in Java. The goal is not to learn Java the language deeply on day one - it's to become fluent enough in Java syntax that DSA problem solving feels as fast as it did in C++.

Table of Contents

1. Mindset Shift: What Changes and What Doesn't

The good news first: **algorithmic thinking does not change**. Big-O analysis, the way you design a two-pointer solution, a sliding window, a graph traversal, or a DP table - all of that transfers 100%. What changes is purely **syntax and library API**.

What stays the same

- Core control flow: if/else, for, while, switch - almost identical syntax.
- Algorithm design: two pointers, sliding window, recursion, backtracking, DP, greedy.
- Big-O reasoning and complexity of standard operations (mostly the same, with a few exceptions noted later).
- Operator precedence and most arithmetic/logical operators (&&, ||, ==, etc.).

What changes

- Everything must live inside a **class** - there's no free-floating main() function like in C++.
- No pointers and no manual memory management - Java has references + garbage collection.
- STL containers become **Java Collections** (java.util package) with different method names.
- No operator overloading (mostly) - so no cout << obj, no + for custom objects unless you write a method.
- Everything you 'new' lives on the heap; there is no stack-allocated object like C++'s vector<int> v(10) on stack.
- Java has no default pass-by-reference for primitives - objects are passed as references, not by value copies.

● *Rule of thumb while transitioning: for every STL container/algorithm you reach for out of habit, ask 'what's the java.util equivalent?' Section 10-14 give you that lookup table.*

2. Program Structure, Variables & Data Types

Minimal program skeleton

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int x = 5;
    cout << x << endl;
    return 0;
}
```

Java

```java
public class Main {
    public static void main(String[] args) {
        int x = 5;
        System.out.println(x);
    }
}
```

Key differences: Java has no header includes for basic types; the class name usually must match the filename (Main.java); execution always starts from `public static void main(String[] args)`.

Primitive data types

C++ Type	Java Type	Size / Notes
int	`int`	32-bit in both. Java's int is ALWAYS 32-bit (not platform dependent).
long long	`long`	64-bit. Java has no 'long long' - just 'long'.
float	`float`	32-bit floating point.
double	`double`	64-bit floating point (default for decimals in both languages).
char	`char`	C++: 1 byte (ASCII). Java: 2 bytes (UTF-16, so 'char' can hold Unicode).
bool	`boolean`	Same concept, different keyword.
unsigned int	`-- (none)`	Java has NO unsigned types at all. Simulate with long or bit tricks.
auto	`var (Java 10+)`	Local type inference. var x = 5; infers int.
size_t	`int / long`	Just use int or long; Java array .length returns int.

● *Biggest gotcha: Java has no unsigned integers. If you rely on unsigned overflow tricks in competitive C++, you'll need long or BigInteger in Java instead.*

Declaring variables

Java

```
int a = 10;
long b = 10000000000L;      // note the L suffix
double d = 3.14;
char c = 'x';
boolean flag = true;
var list = new ArrayList<Integer>(); // type inference, Java 10+
final int CONST = 100;      // "final" = C++ "const"
```

Constants

C++ `const int X = 5;` becomes Java `final int X = 5;`. `#define` macros have no Java equivalent - use `static final` fields instead.

3. Input / Output

This is one of the first practical shocks. C++'s cin/cout are replaced by Scanner (simple but slower) or BufferedReader (fast, used for competitive programming with large input).

Basic I/O

C++

```
int n;
cin >> n;
vector<int> a(n);
for (int i = 0; i < n; i++) cin >> a[i];
cout << "sum = " << n << endl;
```

Java (Scanner - simple, slower)

```
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int[] a = new int[n];
for (int i = 0; i < n; i++) a[i] = sc.nextInt();
System.out.println("sum = " + n);
```

Java (BufferedReader - fast, preferred for competitive programming)

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
int n = Integer.parseInt(br.readLine().trim());
StringTokenizer st = new StringTokenizer(br.readLine());
int[] a = new int[n];
for (int i = 0; i < n; i++) a[i] = Integer.parseInt(st.nextToken());

// Fast output
StringBuilder sb = new StringBuilder();
sb.append("sum = ").append(n).append("\n");
System.out.print(sb);
```

● *Scanner is roughly comparable to un-optimized cin/cout. BufferedReader + StringTokenizer is the Java equivalent of ios::sync_with_stdio(false); cin.tie(NULL); - use it for large inputs on platforms like Codeforces/LeetCode-style constraints.*

4. Operators, Control Flow & Loops

Almost a direct copy-paste. A few important exceptions:

Feature	C++	Java
Integer division	`7 / 2 == 3`	`7 / 2 == 3 (same)`
Modulo with negatives	`-7 % 3 == -1`	`-7 % 3 == -1 (same behavior)`
Ternary	`cond ? a : b`	`cond ? a : b (identical)`
Switch on String	`Not directly (needs if-else or hashing)`	`switch(str) works natively in Java`
for-each	`for (auto x : v)`	`for (int x : list) (identical concept)`

Feature	C++	Java
Bitwise ops	`&, |, ^, ~, <<, >>`	`Same, plus Java has >>> (unsigned right shift)`
goto	`Supported`	`Not supported (use labeled break/continue instead)`

Labeled break (Java's alternative to goto for breaking nested loops)

```
outer:
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        if (grid[i][j] == 0) break outer;
    }
}
```

5. Functions -> Methods

In Java every function is a **method** and must belong to a class. There are no standalone functions like in C++.

C++

```
int add(int a, int b) {
    return a + b;
}

int main() {
    cout << add(2,3);
}
```

Java

```
public class Main {
    static int add(int a, int b) {
        return a + b;
    }
    public static void main(String[] args) {
        System.out.println(add(2, 3));
    }
}
```

- Use `static` methods when you don't need an object instance - this is what you'll do 90% of the time while solving DSA problems (like LeetCode's Solution class methods).
- Java has no default arguments and no function overloading via templates, but it DOES support classic method overloading (same name, different parameter list).
- No function pointers, but Java has **lambdas** and method references (covered in Section 13).

6. Arrays: C++ vs Java

Operation	C++	Java
Fixed array	`int a[5]; or int a[] = {1,2,3};`	`int[] a = new int[5]; or int[] a = {1,2,3};`
Get length	`sizeof(a)/sizeof(a[0]) (only for stack arrays)`	`a.length (property, no parentheses)`
Default values	`Garbage (uninitialized) on stack`	`Always zero/false/null - Java zero-initializes`
Resize	`Not possible for raw arrays`	`Not possible either - arrays are fixed size in both`
Copy array	`std::copy / memcpy`	`Arrays.copyOf(a, a.length) or System.arraycopy(...)`
Fill array	`fill(a, a+n, val)`	`Arrays.fill(a, val)`
Sort array	`sort(a, a+n)`	`Arrays.sort(a)`
Print array	`loop manually`	`Arrays.toString(a) for a quick debug print`

● *Java arrays are objects on the heap even for primitives, so a.length is a field access, not a sizeof trick. There is no stack-array performance difference to worry about in typical DSA use.*

Java array basics

```
int[] a = new int[5];
int[] b = {1, 2, 3, 4, 5};
System.out.println(a.length);      // 5
Arrays.fill(a, -1);
Arrays.sort(b);
int[] c = Arrays.copyOf(b, b.length);
System.out.println(Arrays.toString(b));
```

7. Strings: Mutability Matters

This is a very common source of bugs for people moving from C++. In C++, `std::string` is mutable - you can do `s[i]='x';` directly. In Java, `String` is **immutable**. Every modification creates a new String object.

Task	C++ (std::string)	Java
Concatenate in a loop	`s += c; (efficient, mutable)`	`Use StringBuilder, NOT String += in a loop (O(n^2) otherwise)`
Change one character	`s[i] = 'x';`	`Not allowed directly - convert to char[] or use StringBuilder.setCharAt(i, 'x')`

Task	C++ (std::string)	Java
Substring	`s.substr(start, len)`	`s.substring(start, end)` (note: end index, not length!)
Compare	`s1 == s2` (value compare)	`s1.equals(s2)` -- NEVER use == for String content comparison
Length	`s.length()` or `s.size()`	`s.length()` (no .size() for String)
To char array	already indexable: `s[i]`	`s.toCharArray()` or `s.charAt(i)`
Reverse	`reverse(s.begin(), s.end())`	`new StringBuilder(s).reverse().toString()`

● *The #1 Java bug for C++ transitioners: comparing strings with == compares object references, not content. Always use .equals() (or .equalsIgnoreCase()).*

Java StringBuilder pattern (use this instead of String += in loops)

```
StringBuilder sb = new StringBuilder();
for (char c : "hello".toCharArray()) {
    sb.append(c);
}
sb.reverse();
sb.setCharAt(0, 'H');
String result = sb.toString();
```

8. Pointers, References & Pass-by-Value in Java

- Java has **no pointers** and no pointer arithmetic. No *, no ->, no new/delete pairs to manage manually.
- Java has **no manual memory management** - the Garbage Collector reclaims unused objects automatically.
- Java is **always pass-by-value** - but for objects, the 'value' being passed is a reference (memory address) to the object. So mutating an object's fields inside a method IS visible to the caller, but reassigning the parameter itself is NOT visible to the caller.
- There is no equivalent of C++ passing a primitive by reference (int&) - Java primitives (int, char, boolean...) are always copied. To 'return multiple values', use an array, a small wrapper class, or return a custom object/List.

Why this matters

```
void modify(int[] arr) {
    arr[0] = 99;      // VISIBLE outside - same array object is modified
}

void reassign(int[] arr) {
    arr = new int[]{1,2,3}; // NOT visible outside - only local reference changes
}

void modifyPrimitive(int x) {
    x = 100;          // NOT visible outside - int is always copied
}
```

● *If you're used to passing &x; in C++ to mutate a value in-place, in Java you either wrap it in a 1-element array (int[] holder = {0};), a mutable object, or restructure to return the value.*

9. OOP Differences (Classes, Inheritance, Interfaces)

Feature	C++	Java
Multiple inheritance	Supported (classes can extend multiple classes)	Not supported for classes - only single inheritance (extends one class)
Interfaces	Simulated via abstract classes with pure virtual functions	First-class 'interface' keyword; a class can implement many interfaces
Access modifiers	public / private / protected (before members)	Same 3, plus default 'package-private' if omitted
Virtual functions	virtual keyword required for overriding to work	All non-static methods are virtual by default (overridable)

Feature	C++	Java
Destructors	`~ClassName() { }` manual cleanup	No destructors - use try/finally or AutoCloseable if needed
Struct	`struct` (members public by default)	No struct keyword - just use a class (or a 'record' in Java 16+)
Operator overloading	Supported (+, ==, <<, etc.)	Not supported except '+' for String concatenation

A simple DSA-style class in Java (equivalent of a C++ struct with a constructor)

```
class Point {
    int x, y;
    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
}

// Java 16+ shortcut - equivalent "record" (immutable data holder)
record PointR(int x, int y) {}
```

10. STL to Java Collections - The Big Map

This is the table you'll come back to the most. Import everything from `java.util.*`.

C++ STL	Java Collection	Notes
vector	`ArrayList`	Dynamic array. T must be a wrapper (Integer, not int).
list (doubly linked list)	`LinkedList`	Also implements Deque, List.
deque	`ArrayDeque / LinkedList`	ArrayDeque is generally faster - use it.
stack	`Deque (via ArrayDeque) or Stack`	Prefer ArrayDeque; legacy Stack class is slower (synchronized).
queue	`Queue (via LinkedList or ArrayDeque)`	Use offer()/poll()/peek().
priority_queue	`PriorityQueue`	Java PQ is MIN-heap by default (C++ is MAX-heap by default!)
set	`TreeSet`	Sorted, unique elements. O(log n) ops.
unordered_set	`HashSet`	No ordering guarantee. O(1) avg ops.
multiset	`-- none built-in --`	Simulate with TreeMap (value = count) or a HashMap counter.
map	`TreeMap`	Sorted by key. O(log n) ops.
unordered_map	`HashMap`	No order guarantee. O(1) avg ops.
multimap	`-- none built-in --`	Simulate with Map> using computeIfAbsent.
pair	`-- none built-in --`	Use int[] (for 2 ints), Map.Entry, or a custom class/record.
bitset	`BitSet`	java.util.BitSet - similar API (set/get/flip).
array	`T[] (fixed array)`	Or Collections.unmodifiableList for immutability.

● *The single most dangerous gotcha in this table: C++ priority_queue is a MAX-heap by default, Java's PriorityQueue is a MIN-heap by default. Getting this backwards silently breaks logic instead of throwing an error.*

11. Container-by-Container Deep Dive

11.1 vector -> ArrayList

C++

```
vector<int> v;
v.push_back(5);
v.pop_back();
v[0] = 10;
int sz = v.size();
sort(v.begin(), v.end());
bool empty = v.empty();
```

Java

```
ArrayList<Integer> v = new ArrayList<>();
v.add(5);
v.remove(v.size() - 1);
v.set(0, 10);
int sz = v.size();
Collections.sort(v);
boolean empty = v.isEmpty();
```

● *Java collections cannot hold primitives directly - use wrapper classes (Integer, Long, Character, Double, Boolean). Autoboxing/unboxing happens automatically most of the time.*

11.2 stack -> Deque (as a stack)

C++

```
stack<int> st;
st.push(5);
st.pop();
int top = st.top();
bool empty = st.empty();
```

Java

```
Deque<Integer> st = new ArrayDeque<>();
st.push(5);
st.pop();
int top = st.peek();
boolean empty = st.isEmpty();
```

11.3 queue -> Queue

C++

```
queue<int> q;
q.push(5);
q.pop();
int front = q.front();
bool empty = q.empty();
```

Java

```
Queue<Integer> q = new ArrayDeque<>();
q.offer(5);
q.poll();
int front = q.peek();
boolean empty = q.isEmpty();
```

11.4 priority_queue -> PriorityQueue

C++ (max-heap by default)

```
priority_queue<int> pq;          // max-heap
pq.push(5);
int top = pq.top();
pq.pop();

// min-heap version:
priority_queue<int, vector<int>, greater<int>> minPq;
```

Java (min-heap by default)

```
PriorityQueue<Integer> pq = new PriorityQueue<>(); // min-heap
pq.offer(5);
int top = pq.peek();
pq.poll();

// max-heap version:
PriorityQueue<Integer> maxPq = new PriorityQueue<>(Collections.reverseOrder());
```

11.5 set / map -> TreeSet / TreeMap (sorted)

C++

```
set<int> s;
s.insert(5);
s.erase(5);
bool has = s.count(5);

map<string,int> m;
m["a"] = 1;
for (auto& [k,v] : m) cout << k << v;
```

Java

```
TreeSet<Integer> s = new TreeSet<>();
s.add(5);
s.remove(5);
boolean has = s.contains(5);

TreeMap<String,Integer> m = new TreeMap<>();
m.put("a", 1);
for (Map.Entry<String,Integer> e : m.entrySet())
    System.out.println(e.getKey() + e.getValue());
```

11.6 unordered_set / unordered_map -> HashSet / HashMap

C++

```
unordered_map<string,int> freq;
freq["apple"]++;
if (freq.find("apple") != freq.end()) { }
```

Java

```
HashMap<String,Integer> freq = new HashMap<>();
freq.put("apple", freq.getOrDefault("apple", 0) + 1);
if (freq.containsKey("apple")) { }
// merge() is the idiomatic one-liner:
freq.merge("apple", 1, Integer::sum);
```

● *getOrDefault() and merge() are two of the most useful HashMap methods for DSA - learn them well, they replace a lot of if-checks you'd write in C++.*

11.7 multiset -> TreeMap-based counter

Java - simulating multiset

```
TreeMap<Integer,Integer> multiset = new TreeMap<>();
multiset.merge(5, 1, Integer::sum);      // insert 5
multiset.merge(5, -1, Integer::sum);     // erase one 5
if (multiset.get(5) != null && multiset.get(5) == 0) multiset.remove(5);
int smallest = multiset.firstKey();
int largest  = multiset.lastKey();
```

12. pair<> and tuple<> in Java

Java has no built-in pair or tuple type. Three common workarounds, ranked by how often you'll use them in DSA:

Option 1: int[] (fastest, when both values are int)

Java

```
int[] p = {row, col};
queue.offer(new int[]{r, c});
int r2 = p[0], c2 = p[1];
```

Option 2: int[] arrays of size 2/3 inside a List (for coordinate lists, edge lists)

Java

```
List<int[]> edges = new ArrayList<>();
edges.add(new int[]{u, v, weight});
```

Option 3: a tiny custom class or record (for mixed types / readability)

Java

```
record Pair<A, B>(A first, B second) {}

Pair<String,Integer> p = new Pair<>("apple", 3);
System.out.println(p.first() + " " + p.second());
```

Option 4: Map.Entry (built-in, no custom class needed)

Java

```
Map.Entry<String,Integer> e = Map.entry("apple", 3);
System.out.println(e.getKey() + " " + e.getValue());
```

● *For DSA interviews/competitive practice, int[] is by far the most common because it's fast and avoids boxing overhead in tight loops (e.g., BFS with (row, col) pairs).*

13. Custom Sorting: Comparator vs Comparable

C++'s custom comparator function/lambda passed to sort() maps to Java's **Comparator** interface, usually written as a lambda.

C++

```
vector<pair<int,int>> v = {{1,5},{2,3}};
sort(v.begin(), v.end(), [](auto &a, auto &b) {
    return a.second < b.second;   // sort by second value asc
});
```

Java

```
List<int[]> v = new ArrayList<>();
v.add(new int[]{1,5});
v.add(new int[]{2,3});
v.sort((a, b) -> a[1] - b[1]);   // sort by index 1 asc
// or: Collections.sort(v, Comparator.comparingInt(a -> a[1]));
```

Multi-key sorting

Java - sort by column 0 ascending, tie-break by column 1 descending

```
v.sort((a, b) -> {
    if (a[0] != b[0]) return a[0] - b[0];
    return b[1] - a[1];
});

// Fluent style using Comparator chaining:
v.sort(Comparator.<int[]>comparingInt(a -> a[0])
        .thenComparing(a -> -a[1]));
```

Making a custom class sortable by default (like overloading operator< in C++)

Java - implement Comparable for natural ordering

```
class Person implements Comparable<Person> {
    String name; int age;
    Person(String name, int age) { this.name = name; this.age = age; }

    @Override
    public int compareTo(Person other) {
        return this.age - other.age;   // natural order = by age ascending
    }
}

List<Person> people = new ArrayList<>();
Collections.sort(people);   // uses compareTo automatically
```

● *Comparable = 'this class has ONE natural sort order' (like operator< overload in C++). Comparator = 'here is a specific sort order I want right now' (like a custom comparator passed to sort()). You can define many Comparators but only one compareTo() per class.*

14. STL -> Java Equivalents

C++	Java Equivalent
sort(v.begin(), v.end())	`Collections.sort(list) or Arrays.sort(array)`
reverse(v.begin(), v.end())	`Collections.reverse(list) (arrays: reverse manually or via a helper)`
max(a, b) / min(a, b)	`Math.max(a, b) / Math.min(a, b)`
*max_element(v.begin(), v.end())	`Collections.max(list)`
*min_element(v.begin(), v.end())	`Collections.min(list)`
accumulate(v.begin(), v.end(), 0)	`Manual loop, or list.stream().mapToInt(Integer::intValue).sum()`
binary_search(v.begin(), v.end(), x)	`Collections.binarySearch(list, x) or Arrays.binarySearch(arr, x)`
lower_bound(v.begin(), v.end(), x)	`TreeSet.ceiling(x) or a manual binary search (no direct 1:1 for arrays)`
upper_bound(v.begin(), v.end(), x)	`TreeSet.higher(x) or a manual binary search`
swap(a, b)	`Collections.swap(list, i, j) (for arrays: manual 3-line swap - no built-in for primitives)`
unique(v.begin(), v.end())	`new LinkedHashSet<>(list) (dedupe while roughly preserving order)`
next_permutation(v.begin(), v.end())	`-- none built-in -- (write manually or use recursion for permutations)`
fill(v.begin(), v.end(), x)	`Collections.fill(list, x) or Arrays.fill(array, x)`
__gcd(a, b)	`Math.floorDiv / write gcd manually, or use BigInteger.gcd for large numbers`

● *Java has no next_permutation and no direct lower_bound/upper_bound for plain arrays - these are the two STL habits that require the most rewriting when you switch.*

15. 2D Arrays & Vector of Vectors

C++

```
vector<vector<int>> grid(n, vector<int>(m, 0));
grid[2][3] = 7;
int rows = grid.size();
int cols = grid[0].size();
```

Java

```
int[][] grid = new int[n][m];         // all zeros by default
grid[2][3] = 7;
int rows = grid.length;
int cols = grid[0].length;

// If you need a resizable 2D structure:
List<List<Integer>> grid2 = new ArrayList<>();
for (int i = 0; i < n; i++) grid2.add(new ArrayList<>());
```

Both languages default-initialize 2D int arrays to 0, so you rarely need an explicit fill loop in either. Use int[][] whenever the dimensions are fixed and known upfront - it's faster than List<List<Integer>> due to no boxing.

16. Full Worked Examples

16.1 BFS on a grid

C++

```
int bfs(vector<vector<int>>& grid, int sr, int sc) {
    int n = grid.size(), m = grid[0].size();
    vector<vector<bool>> vis(n, vector<bool>(m, false));
    queue<pair<int,int>> q;
    q.push({sr, sc});
    vis[sr][sc] = true;
    int steps = 0;
    int dr[] = {0,0,1,-1}, dc[] = {1,-1,0,0};
    while (!q.empty()) {
        int sz = q.size();
        for (int i = 0; i < sz; i++) {
            auto [r,c] = q.front(); q.pop();
            for (int d = 0; d < 4; d++) {
                int nr = r+dr[d], nc = c+dc[d];
                if (nr>=0 && nr<n && nc>=0 && nc<m && !vis[nr][nc] && grid[nr][nc]==0) {
                    vis[nr][nc] = true;
                    q.push({nr, nc});
                }
            }
        }
        steps++;
    }
    return steps;
}
```

Java

```
static int bfs(int[][] grid, int sr, int sc) {
    int n = grid.length, m = grid[0].length;
    boolean[][] vis = new boolean[n][m];
    Queue<int[]> q = new ArrayDeque<>();
    q.offer(new int[]{sr, sc});
    vis[sr][sc] = true;
    int steps = 0;
    int[] dr = {0,0,1,-1}, dc = {1,-1,0,0};
    while (!q.isEmpty()) {
        int sz = q.size();
        for (int i = 0; i < sz; i++) {
            int[] cur = q.poll();
            int r = cur[0], c = cur[1];
            for (int d = 0; d < 4; d++) {
                int nr = r + dr[d], nc = c + dc[d];
                if (nr>=0 && nr<n && nc>=0 && nc<m && !vis[nr][nc] && grid[nr][nc]==0) {
                    vis[nr][nc] = true;
                    q.offer(new int[]{nr, nc});
                }
            }
        }
        steps++;
    }
    return steps;
}
```

16.2 Dijkstra-style shortest path with PriorityQueue

Java

```
static int[] dijkstra(List<List<int[]>> adj, int src, int n) {
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[src] = 0;
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]); // min-heap by dist
    pq.offer(new int[]{src, 0});
    while (!pq.isEmpty()) {
        int[] cur = pq.poll();
        int u = cur[0], d = cur[1];
        if (d > dist[u]) continue;
        for (int[] edge : adj.get(u)) {
            int v = edge[0], w = edge[1];
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.offer(new int[]{v, dist[v]});
            }
        }
    }
    return dist;
}
```

16.3 Frequency count + sort by frequency (very common interview pattern)

Java

```
static List<Integer> topKFrequent(int[] nums, int k) {
    HashMap<Integer,Integer> freq = new HashMap<>();
    for (int x : nums) freq.merge(x, 1, Integer::sum);

    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]); // min-heap by freq
    for (Map.Entry<Integer,Integer> e : freq.entrySet()) {
        pq.offer(new int[]{e.getKey(), e.getValue()});
        if (pq.size() > k) pq.poll();
    }
    List<Integer> result = new ArrayList<>();
    while (!pq.isEmpty()) result.add(pq.poll()[0]);
    Collections.reverse(result);
    return result;
}
```

17. Common Pitfalls Moving from C++ to Java

- **Comparing objects with ==** - for String and any wrapper/custom object, == checks reference identity, not value equality. Always use .equals(). (Small cached Integers -128 to 127 may accidentally == correctly - don't rely on this.)
- **Forgetting autoboxing costs** - ArrayList<Integer> boxes/unboxes every int. In tight loops on large inputs, prefer int[] arrays over ArrayList<Integer> for performance.
- **PriorityQueue is min-heap by default** - opposite of C++'s priority_queue. Pass Collections.reverseOrder() or a custom comparator for a max-heap.
- **Integer overflow** - Java int is always 32-bit like C++ int; if you used 'long long' habitually in C++ for safety, use 'long' in Java the same way.
- **ConcurrentModificationException** - modifying a List/Set/Map while iterating over it with a for-each loop throws this at runtime. Use an Iterator's remove() method, or iterate over a copy.
- **No default toString() for arrays** - System.out.println(array) prints a memory-hash-like string, not the contents. Use Arrays.toString(array) or Arrays.deepToString(matrix).
- **Recursion depth** - Java's default stack size is often smaller than what competitive C++ setups use; very deep recursion (e.g., 10^5+ depth) can StackOverflowError faster than in C++. Consider converting to iterative solutions for very deep recursion.
- **Static vs instance context** - if your main() is static, any method it calls directly must also be static (or called on an instance) - a very common first compile error.

18. Practice Roadmap

- **Day 1-2:** Re-write 10 easy problems you've already solved in C++ (arrays, strings, hashmap-based) purely in Java, using this guide as a lookup sheet.
- **Day 3-4:** Focus specifically on Collections - redo problems using ArrayList, HashMap, HashSet, TreeMap, and Deque until the method names feel automatic (add/get/put/offer/poll/peek).
- **Day 5:** Redo 3-4 heap/priority-queue problems, paying close attention to min-heap vs max-heap direction.
- **Day 6:** Redo 2-3 custom-sorting problems using Comparator lambdas and Comparator.thenComparing chains.
- **Day 7:** Redo a graph problem (BFS/DFS/Dijkstra) fully in Java without referencing your old C++ code, then compare against Section 16 of this guide.
- **Ongoing:** Whenever you reach for an STL container/algorithm out of muscle memory, look it up in Sections 10 and 14 instead of guessing - within ~2 weeks of consistent practice this becomes automatic.

Good luck with the transition - your DSA fundamentals from C++ are the hard part, and you've already done that. This is just new vocabulary for the same ideas.